



Guia de Configuração: Memória Unificada de IA no Windows 11 (Estilo Apple Silicon)

Este guia descreve como o **Unified Memory Coordinator (UMC)** unifica as tuas placas gráficas (GPU 0 + GPU 1) com a System RAM de forma matemática e automática para correres modelos gigantes no ComfyUI, Stable Diffusion e Ollama sem nunca veres o erro *Out of Memory (OOM)*.

1. A Matemática da Unificação (RAID 0 + Aperture Segment)

No Apple Silicon (M1/M2/M3 Max), a CPU e a GPU partilham o mesmo barramento físico. No teu setup Windows 11 (Ryzen 9 5950X + Multi-GPU), emulamos este comportamento criando um **Aperture Segment** virtual via barramento PCIe Gen 4.

Distribuição Circular de Blocos (Striping RAID 0)

Para maximizar a largura de banda, os tensores do modelo de IA (pesos de difusão e matrizes de atenção) são divididos em blocos lógicos de tamanho constante S_{block} (ex: 4 MB, 8 MB).

Cada bloco com um ID único B_{id} é distribuído pelas GPUs disponíveis usando a fórmula de Stride circular:

$$I_{target} = B_{id} \bmod N_{gpus}$$

Onde:

- I_{target} é o índice da GPU onde o bloco será armazenado.
- N_{gpus} é o número de GPUs CUDA ativas no sistema.

Se a GPU selecionada atingir o seu limite seguro de VRAM (85% da capacidade máxima para evitar colisões com o DWM do Windows), o UMC aciona imediatamente o **Tensor Eviction Manager (LRU)**, movendo os blocos menos acedidos recentemente para a RAM de alta velocidade em modo Pinned:

$$T_{access} = \min_{block \in \text{VRAM}} (LastAccessTime_{block})$$

2. Como Ligar a UMC ao PyTorch (ComfyUI / Stable Diffusion)

O PyTorch utiliza por padrão o seu próprio alocador de memória CUDA (`caching_allocator`). Para forçarmos o PyTorch a usar o nosso driver de Memória Unificada em C++, usamos as APIs nativas de interceptação e preload.

Passos de Configuração do Ambiente do Windows:

1. **Memória Não-Paginável (Pinned Memory):** O driver UMC pré-aloca um pool de memória física na RAM usando `cudaHostAlloc` com mapeamento de barramento ativo. Isto permite que a tua GPU leia dados da RAM a velocidades que atingem o limite teórico do teu barramento PCIe Gen 4:

$$\text{Largura de Banda Teórica (PCIe 4.0 x16)} \approx 31.5 \text{ GB/s}$$

2. **Configuração de Variáveis de Ambiente:** Deves configurar o PyTorch para não tentar gerir a memória de forma agressiva e dar prioridade à paginação assíncrona do Windows:
 - `PYTORCH_CUDA_ALLOC_CONF=backend:cudaMallocAsync`
 - `CUDA_MODULE_LOADING=LAZY` (atrasa o carregamento de kernels CUDA não utilizados, poupando até 1 GB de VRAM estática).

3. Fluxo de Execução com o ComfyUI

Ao correr o ComfyUI através do nosso script de arranque unificado:

1. O driver **UMC** monitoriza as tabelas de alocação das tuas GPUs.
2. Durante o passo de amostragem (KSampler), as matrizes do modelo UNet/DiT que não cabem na VRAM são mapeadas diretamente para a RAM Pinned.
3. Graças à transferência assíncrona e prefetching por canais DMA (`cudaMemcpyAsync`), a GPU continua a computar as matrizes locais enquanto o barramento PCIe puxa os próximos blocos do modelo em background, eliminando gargalos de processamento.